
BLM0212 VERİ YAPILARI • LAB 3

Algoritma Verimliliği ve Big-O

Döngü türleri ve verimlilikleri

Döngü türü	Kontrol değişkeni	Tekrar sayısı	Big-O
Doğrusal	$i++$ veya $i += 2$	n (veya $n/2$)	$O(n)$
Logaritmik	$i *= 2$ veya $i /= 2$	$\log_2 n$	$O(\log n)$
Doğrusal logaritmik	dış $i++$, iç $j *= 2$	$n \cdot \log n$	$O(n \log n)$
Karesel	dış $i++$, iç $j++$ (n'e kadar)	$n \cdot n$	$O(n^2)$
Bağımlı karesel	iç döngü $j < i$	$n(n + 1) / 2$	$O(n^2)$

Kural: Kontrol değişkeni toplama/çıkarma ile değişiyorsa döngü doğrusaldır; çarpma/bölme ile değişiyorsa logaritmiktir.

f(n)'den Big-O'ya

İki adım

1. Her terimin katsayısını 1 yap.
2. En büyük terim dışındakileri at.

$$\begin{aligned}f(n) &= n(n + 1) / 2 \\&= \frac{1}{2}n^2 + \frac{1}{2}n \\&\rightarrow n^2 + n \\&\rightarrow O(n^2)\end{aligned}$$

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

n = 1000 için adım sayısı

f(n)	Yaklaşık değer
log n	10
n	1 000
n log n	10 000
n ²	1 000 000
n ³	10 ⁹
2 ⁿ	≈ 10 ³⁰¹

ÖRNEK 1 · DOĞRUSAL DÖNGÜ

Dizideki en büyük eleman

```
int enBuyuk(int dizi[], int n) {  
    int max = dizi[0];  
    for (int i = 1; i < n; i++)  
        if (dizi[i] > max)  
            max = dizi[i];  
    return max;  
}
```

Döngü kaç kez döner?

i, 1'den $n - 1$ 'e kadar birer artıyor. Döngü gövdesi $n - 1$ kez çalışır.

$$f(n) = n - 1 \rightarrow O(n)$$

i += 2 olsaydı?

Tekrar sayısı yarıya iner: $(n - 1) / 2$. Sabit çarpanlar Big-O'yu değiştirmez, sonuç yine $O(n)$.

ÖRNEK 2 · LOGARITMIK DÖNGÜ

Bir sayının basamak sayısı

```
int basamakSay(int sayi) {  
    int adet = 0;  
    while (sayi > 0) {  
        sayi /= 10;  
        adet++;  
    }  
    return adet;  
}
```

Her turda sayı 10'a bölünüyor. Kontrol değişkeni bölünerek değiştiği için döngü **logaritmiktir**.

sayi (n)	Tekrar
7	1
75	2
1 234	4
1 000 000	7

$$f(n) = \lfloor \log_{10} n \rfloor + 1 \rightarrow O(\log n)$$

ÖRNEK 3 · BAĞIMLI KARESEL DÖNGÜ

Dizide tekrar eden eleman var mı?

```
int tekrarVarMi(int dizi[], int n) {  
    for (int i = 0; i < n - 1; i++)  
        for (int j = i + 1; j < n; j++)  
            if (dizi[i] == dizi[j])  
                return 1;  
    return 0;  
}
```

En kötü durum: dizide tekrar yok, bütün çiftler karşılaştırılır.

i	İç döngü tekrarı
0	n - 1
1	n - 2
...	...
n - 2	1

$$\begin{aligned}f(n) &= n(n - 1) / 2 \\&= \frac{1}{2}n^2 - \frac{1}{2}n \rightarrow O(n^2)\end{aligned}$$

ALİŞTIRMA 1

Ortalamadan büyük elemanlar

Bir int dizisinde, dizinin ortalamasından **büyük** olan eleman sayısını döndüren fonksiyonu yazınız.

```
int ortalamadanBuyukSay(int dizi[], int n)
```

Örnek: {4, 8, 15, 16, 23, 42} → ortalama 18 → sonuç 2

Kodu yazdıktan sonra aşağıdaki soruları cevaplayınız.

1. Kodunuzdaki döngüler kaç kez dönüyor?
2. Kodunuzun $f(n)$ 'i ve Big-O'su nedir?

ALİŞTIRMA 2

2'nin kuvvetleri

1'den n'e kadar (n dahil) **2'nin kuvvetlerini** ekrana yazdıran fonksiyonu yazınız.

```
void ikininKuvvetleri(int n)
```

Örnek: $n = 100 \rightarrow 1\ 2\ 4\ 8\ 16\ 32\ 64$

Kodu yazdıktan sonra aşağıdaki soruları cevaplayınız.

1. $n = 1\ 000$ için kaç sayı yazdırılır?
2. Kodunuzun $f(n)$ 'i ve Big-O'su nedir?

ALİŞTIRMA 3

Yıldız üçgeni

n satırlık bir yıldız üçgeni yazdıran fonksiyonu yazınız. **i. satırda i tane yıldız** olacak.

```
void ucgen(int n)
```

n = 4 için çıktı

*

**

Kodu yazdıktan sonra aşağıdaki soruları cevaplayınız.

1. n = 5 için toplam kaç yıldız yazdırılır?
2. Kodunuzun f(n)'i ve Big-O'su nedir?

BÖLÜM 3 • ~7 DAKIKA

Çözümler

Cevabınızı kontrol edin: tekrar sayısı, $f(n)$ ve Big-O.

Ardışık döngüler toplanır

```
int ortalamadanBuyukSay(int dizi[], int n) {  
    double toplam = 0;  
    for (int i = 0; i < n; i++)  
        toplam += dizi[i];  
    double ort = toplam / n;  
    int adet = 0;  
    for (int i = 0; i < n; i++)  
        if (dizi[i] > ort)  
            adet++;  
    return adet;  
}
```

İki ayrı döngü var ve ikisi de n kez dönüyor. Döngüler iç içe değil, art arda.

$$\begin{aligned} f(n) &= n + n = 2n \\ &\rightarrow O(n) \end{aligned}$$

Neden tek döngü yetmez? Ortalama ancak ilk döngü bitince bilinir.

Çarparak ilerleyen döngü

```
void ikininKuvvetleri(int n) {  
    for (int i = 1; i <= n; i *= 2)  
        printf("%d ", i);  
}
```

Alternatif: 1'den n'e her sayıyı kontrol etmek. Aynı çıktı, ama döngü n kez döner: $O(n)$.

n	Yazdırılan sayı
100	7 (1 ... 64)
1 000	10 (1 ... 512)

$$f(n) = \lfloor \log_2 n \rfloor + 1$$
$$\rightarrow O(\log n)$$

i her turda 2 ile çarpıldığı için döngü logaritmik.

İç döngü dış döngüye bağlı

```
void ucgen(int n) {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= i; j++)  
            printf("*");  
        printf("\n");  
    }  
}
```

Satır (i)	Yıldız
1	1
2	2
...	...
n	n

$$\begin{aligned}f(n) &= 1 + 2 + \dots + n \\ &= n(n + 1) / 2 \rightarrow O(n^2)\end{aligned}$$

n = 5 için 15 yıldız.

Döngü × fonksiyon maliyeti

```
void basamaklariYaz(int n) {  
    for (int i = 1; i <= n; i++)  
        printf("%d: %d\n", i,  
               basamakSay(i));  
}
```

Dış döngü n kez döner. Her turda basamakSay en fazla $\log_{10} n + 1$ adım sürer.

$$\begin{aligned} f(n) &\approx n \cdot \log n \\ &\rightarrow O(n \log n) \end{aligned}$$

Kontrol: $n = 100$ için basamakSay'ın içindeki döngü toplam $9 \cdot 1 + 90 \cdot 2 + 1 \cdot 3 = 192$ kez döner.

ÖZET

Karmaşıklığı bulmak için dört adım

1. Döngüleri say

Verimliliği belirleyen döngülerdir. Gövde kaç kez çalışıyor?

2. Değişkene bak

+ veya – ile değişen kontrol değişkeni doğrusal, $\times$ veya $\div$ ile değişen logaritmik döngü verir.

3. İç içe ise çarp

İç içe döngülerde tekrar sayıları çarpılır. İç döngü dışa bağlıysa turlar tek tek toplanır.

4. Sadeleştir

Big-O için katsayıları at, yalnızca en büyük terimi tut.